

MLOps: Machine Learning Operations

Session 3

ML Pipelines, Environments & Security

Juan Manuel Ramírez García · jramirezg@faculty.ie.edu

Live online session · Saturday 10 October 2026 · 90 minutes

Data ingestion &
versioning

Data validation

Data pre-processing

Model training

Model tuning

Model analysis

Model validation

Model deployment

Model feedback

Where we are: what Forum 1 said, and what today adds

From Forum 1

What you said

- “A trained model is a result; an operational model is a service.” — Marcos
- “Green means up, not right.” — Armen
- Misconception of the week: operationalising = “putting it on a server”

By the end of this session you can...



Turn a notebook into a pipeline

Name the steps and say what each one protects against.



Justify Dev, QA and Prod

And name one way a model can be attacked, with a control.

Minute	Block
0-8	Recap, outcomes, your "messy things", the plan map
8-16	From notebook to pipeline
16-35	Data: questions, versioning, validation (+ 7-min MLflow demo)
35-46	Environments: Dev, QA, Prod (+ 1 poll)
46-51	Break
51-58	Security: how a model can be attacked
58-76	Activity: a risk-mitigation blueprint for a bank
76-86	Debrief
86-90	Three things, your forum, your reading

What you found messy in María's notebook



It cannot be re-run

- "Don't run all cells at once"
- Cells executed out of order (look at the numbers)
- A laptop path, then a relative path
- The scaler cell breaks if run twice



Nobody checks the data

- "Looks fine" is a comment, not a test
- -1 support tickets, 999 charges, "north"
- Duplicated customers, dates as text
- Missing values filled with the mean, mid-way



The result cannot be trusted

- 0.84 yesterday, 0.87 today
- Three copies of the same model, edited by hand
- Threshold chosen on the test set
- Scaler and "churn rate by plan" see the answers



Every messy thing is a production failure waiting to happen. Each column is a part of today.

The questions an MLOps plan must answer; today we take the first four

Access data	Feature engineering	Model training	Model registry	Model deployment	Model monitoring
Data accessibility Data format Data validation	Reproducibility Scalability Training-serving skew	Experiment tracking Reproducibility Scalability	Model versioning Auditing Ownership	Reliability Scalability Endpoint (Session 5)	Logging & observability Data drift Fairness, quality, explicability (Session 7)

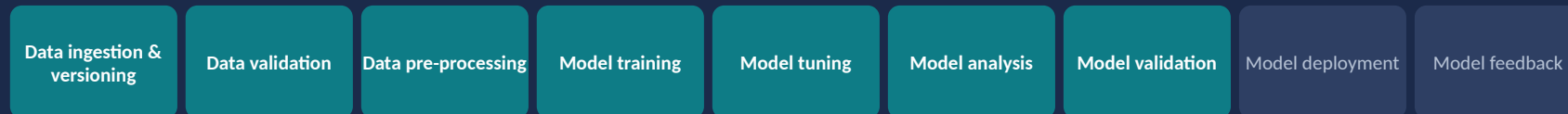
Across all stages: CI/CD and automation (Session 5)

Across all stages: security, audit and governance (today, Sessions 7 and 12)

Your group project is this table, filled in for a real scenario, with a roadmap and owners.

From notebook to pipeline

What a model really is, and why it needs a factory rather than a workshop



The lifecycle we follow through the course · highlighted: where this section sits

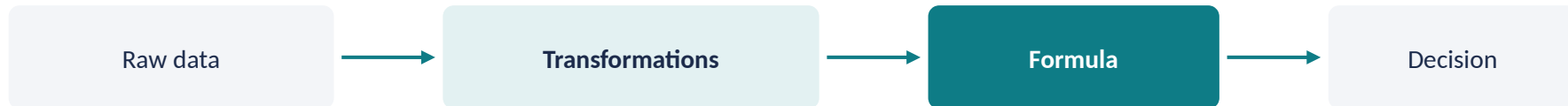
In practice, a model is a formula plus everything needed to feed it

The formula

Once trained, a model is a function: inputs in, a number out. Stateless, deterministic. Whether it is a neural network or a rule, operations treat it the same way.

...plus its transformations

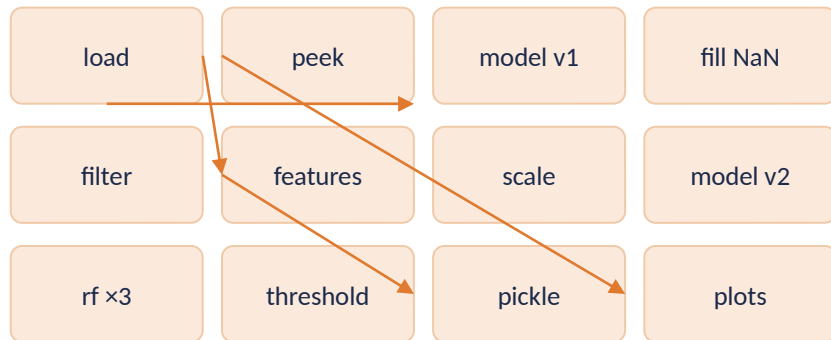
Cleaning, scaling, derived features, encodings. They are part of the model. If production computes them differently, the formula receives numbers it has never seen.



Adapted from Treveil et al., Introducing MLOps (O'Reilly, 2020). In María's notebook the transformations live in 20 scattered cells: that is what cannot be handed to IT.

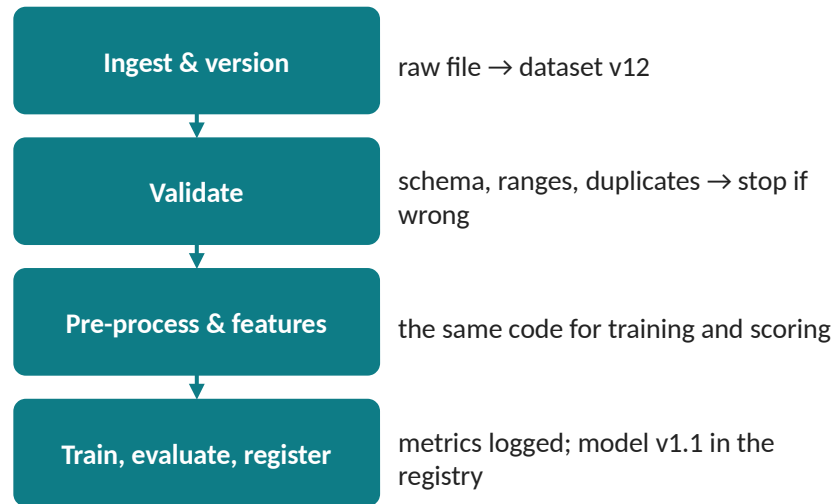
The same work, drawn twice: a notebook and a pipeline

Notebook: one person, one order, one laptop



Cells run out of order; the state lives in memory; "don't run all cells at once".

Pipeline: steps a machine runs, in order, every time



Each step can be tested alone, re-run alone, and read by anyone. Forum 2, Q1: draw this for María's notebook.

Why pipelines: less maintenance, fewer bugs, and a system that runs itself



New data triggers the whole chain

Validate, pre-process, train, analyse, deploy: nobody has to remember the steps.



A whole class of bugs disappears

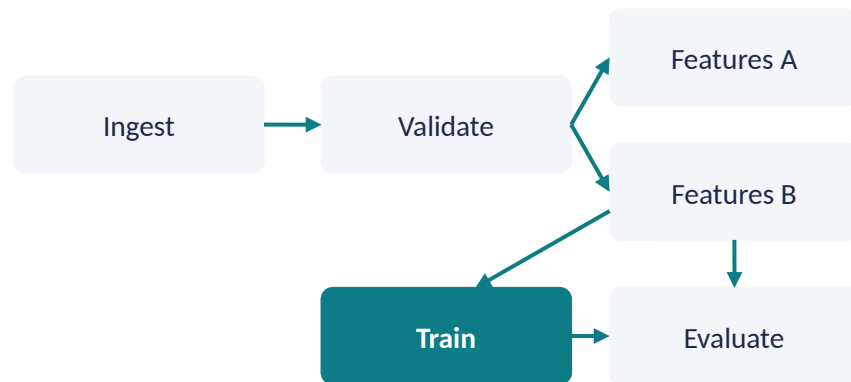
The classic: pre-processing changed after training; the model silently receives different numbers.



Experiments and releases are tracked

Which data, which code, which settings produced which model. Reproducible.

Orchestration: steps in the right order, drawn as a DAG



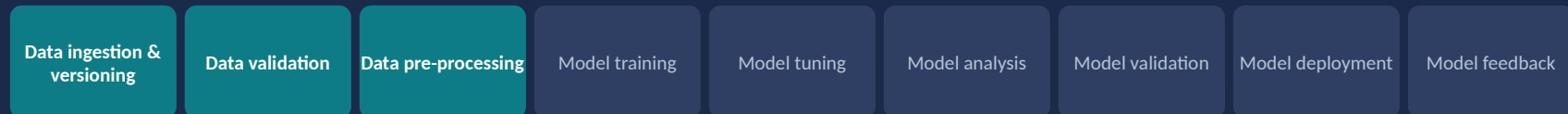
Directed: arrows have a direction. Acyclic: no loops back. A step waits until its inputs exist. Tools: Apache Airflow, Kubeflow Pipelines, cloud-native equivalents.



Source: Hapke & Nelson, *Building Machine Learning Pipelines* (O'Reilly, 2020)

Data: where the time, and the failures, live

Six questions, three checks, and a memory for every dataset



The lifecycle we follow through the course · highlighted: where this section sits

Six questions to answer before the first model is trained



Which datasets exist, and are they reliable?

Where they live, who owns them, how often they are wrong.



Who can access them, and how?

Silos are the norm: the data exists, the permission does not.



Will the same data exist at prediction time?

A feature computed from last month's warehouse is useless for a decision made now.



Do we have the answer we want to predict?

Labels: how many, how late, and what it costs to get more.



How will data be updated once the model is live?

Who tells the data team when a source changes?



Are we allowed to use it?

PII, GDPR, protected attributes such as gender or nationality; representation of minorities.



Which of these did NorthRetail get wrong? (type the number in chat)

The data you train on must be obtainable, in the same shape, when the model runs



Model development

- Data extracted from live systems and warehoused offline
- Cleaned by hand, at leisure, in a notebook
- Months of history, sometimes unstructured (news, documents)



Model in production

- Real-time or nightly data, whatever arrives
- No human in the loop to fix it
- No time to analyse unstructured history fast enough



Rule 1: the data structures must be the same in both places. Rule 2: everything the model needs must be obtainable at the moment of the decision, not tonight.

Version the data like the code: splits, snapshots and a fingerprint



Split once, keep the split

- Training, evaluation, test: decided once, saved
- Random re-splitting is why the notebook gave 0.84 yesterday and 0.87 today



Snapshot the data, number it

- Every hour, day or week, ingestion takes a snapshot ("span")
- "Trained on span 12" is a fact; "trained on the data" is not



Fingerprint it

- Store a hash of the dataset with the model, not only the file name
- Later you can prove which data produced which model



Session 1: a repository is a folder with a memory. This is the same memory, for data. The demo in a few minutes shows where the memory for models lives.

Data validation: three checks that stop the pipeline before it wastes a day

1

Anomalies: are there impossible values?

Negative ticket counts, 999 charges, duplicated customers, 60% empty columns. The notebook printed them and wrote "looks fine".

2

Schema: are the columns what we expect?

Names, types, allowed values, ranges. A supplier changing product codes fails here, loudly, before anything is trained.

3

Statistics: does new data still look like training data?

Distributions compared with the previous span. If they moved a lot, either the world changed or the extract is broken; both need a human.



If the pipeline is going to retrain models on its own, validation is what makes that safe. Tools: TensorFlow Data Validation, Great Expectations, Evidently.

Pre-processing and features: the same code in training and in production



Training-serving skew

- Pre-processing in the notebook $\neq$ pre-processing in the API
- Fix: one pre-processing step, one code, used by both training and scoring



Feature engineering, with a price

- New variables $\rightarrow$ probably more accuracy
- ... $\rightarrow$ more cost, harder to reproduce live, harder to explain

Feature store: one shared, versioned library of features for training and scoring



Concerns: too rigid? too complex for three models? Worth it at forty.

The memory for models: an experiment log and a registry



The experiment log

Every run: code version, data version, settings, metrics, the model file. Nothing is "the one from Tuesday".



The registry

The shortlist. Each model has a version, an owner, a status: production, candidate, retired.



Plus the repository and the data snapshots

Code v2.1 + span 12 + run 4 = model v1.1. Reproducible, auditable.



Live demo • 7 minutes

MLflow: five runs of the churn model side by side, one rejected by validation, and a registry with the version in production and the candidate. Tools: MLflow, Weights & Biases, the cloud platforms.

What you just saw



The runs table

Every attempt in one view: settings, data version, metrics. The best number is not automatically the best model.



The rejected run

Accuracy 0.80 on an extract with 60% empty tenure. The metric barely moved; the validation check caught it.



The registry

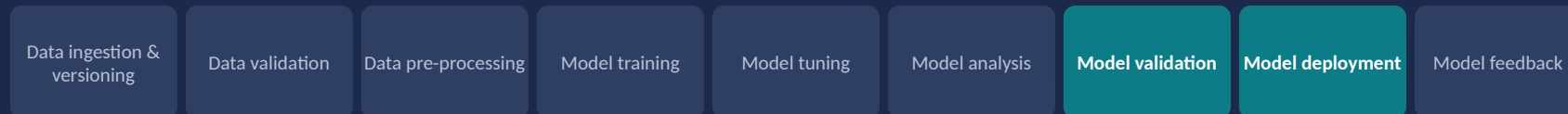
v1 in production, with owner and approval date; v2 a candidate. Registering is cheap; promoting is the gate: a pointer only certain people may move.



The registry says which model is approved. It does not say where it runs. Next: environments.

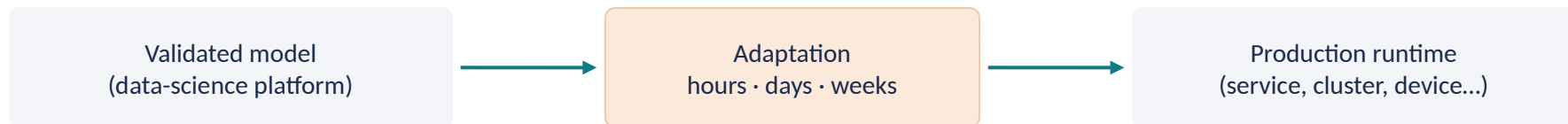
Environments: Dev, QA, Prod

Why companies keep three copies of everything, and what it costs to skip one



The lifecycle we follow through the course · highlighted: where this section sits

Before "where", "whether": can the model run there at all?



Tooling

Built in Python; production expects a Java-compatible format (PMML, ONNX).



Performance

Python is slow per record; the model must run on a low-power device.



Data access

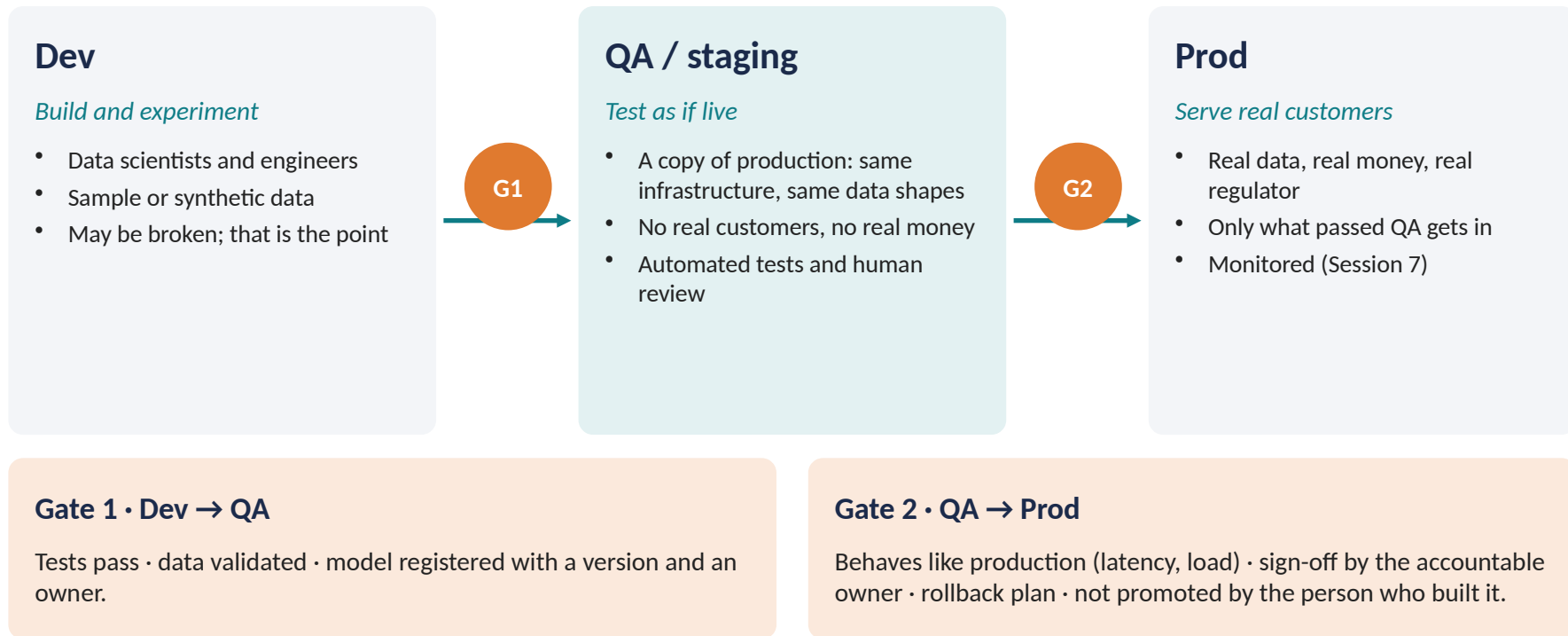
The features must be reachable from where the model runs, at decision time.



Ideal: the model validated in development goes to production as is. Lesson: decide the production format on day one, not at the end.

Adapted from Treveil et al., Introducing MLOps (O'Reilly, 2020)

Three environments, two gates: Dev, QA, Prod



What goes wrong when a model goes from a laptop straight to production



Different libraries

The laptop had version 1.3; production has 1.1. The model loads, and predicts nonsense.



Pre-processing left behind

The transformations lived in cells that were never shipped.



Never tested under load

Works for one row; falls over at a thousand per second.



Tested on a file, not the feed

The live data has different column names and arrives late.



Nothing to roll back to

No version, no tag, no previous model kept.



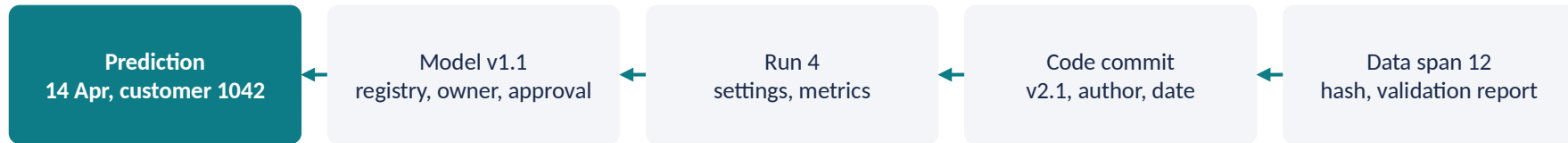
Nobody signed off

So nobody is accountable when it fails on Saturday.



Chat poll (2 min): "A model that works on the data scientist's laptop can go to production the same day if the business is in a hurry." AGREE or DISAGREE, one line why.

Reproducibility, auditability, lineage: one trace from a prediction back to its data



← walk backwards from any prediction



Reproducibility

- Re-run the exact same experiment: same data, same code, same environment
- An artefact that bundles model + environment specification



Auditability

- Full history for every version, in one reliable place
- Documentation, tests, explanations, fairness reports, logs



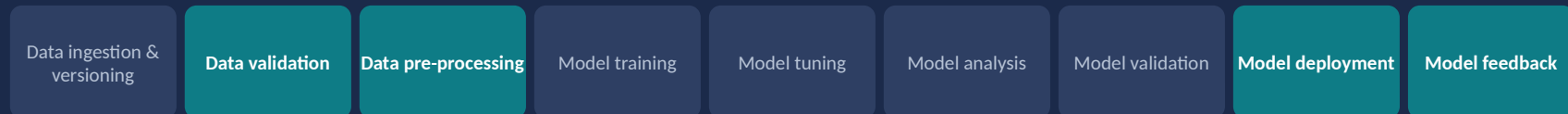
Lineage is also how you detect an attack or a broken feed after the fact: which snapshot and which change entered on the day predictions went wrong? (Forum 2, Q3)

Break • 5 minutes

Back at minute 51. While you wait: what is the most valuable model in your company, and who could break it on purpose?

Security: how a model can be attacked, and how to protect it

Where model risk comes from, four kinds of attack, and the controls that answer them



The lifecycle we follow through the course · highlighted: where this section sits

Where model risk comes from



Sources of risk

- Bugs in design, training or data preparation
- Bugs in the runtime, or in the conversion between environments
- Low-quality training data; production data unlike training data
- Expected error rate, unexpected consequences
- Misuse or misinterpretation of outputs
- Adversarial attacks
- Legal liability; reputational damage from bias



Amplified by

- Broad use of the model
- A rapidly changing environment
- Complex interactions between models



A catalogue to start from

- IBM AI Risk Atlas: risks by input, training, inference, output, non-technical
- Use it in the group project as the checklist

Adapted from Treveil et al., Introducing MLOps (O'Reilly, 2020) and IBM watsonx documentation

Four ways to attack a model, each with a picture and a control



Data poisoning

Corrupt the training data so the model learns the wrong thing.

Control: Validation, provenance of labels, lineage



Adversarial input (evasion)

Craft inputs the model misreads: transactions just below the threshold.

Control: Monitor probing patterns, rate limits, retrain, never expose raw scores



Model extraction & inversion

Query the API many times to rebuild the model or recover training data.

Control: Authentication, rate limits, output rounding, query monitoring



Prompt injection (LLMs)

Instructions hidden in the input hijack the model. Session 10.

Control: Input filtering, output guardrails, least privilege



Public catalogues: OWASP Machine Learning Top 10 · MITRE ATLAS (adversarial threat landscape for AI systems)

Controls: the ones you already have, and the ones AI adds



Information security (existing)

- Access control and authentication
- Encryption of data at rest and in transit
- Backups, data destruction
- Incident registration and treatment
- Audit plans and reviews



AI-specific

- Data validation and label provenance
- Drift detection and retraining
- Endpoint protection: rate limits, output rounding, injection filters
- Security of training datasets



AI-system level

- Register of AI systems and owners
- Risk assessment per use case
- Data governance: lineage, catalogue
- Model monitoring, bias checks
- Accountability: who signs off



In the activity you will pick one control per threat, name who owns it, and say how you would detect the threat. Your answers come from these three columns.

Activity • 15 minutes in breakout rooms: a model-risk mitigation blueprint



Your task (15 min)

- Read the one-page case: IberBank card-fraud model, three incidents
- For each incident, fill a row of the blueprint: threat • lifecycle stage • one control • who owns it • how you would detect it
- Bonus: which incident would a regulator care about most, and why?



How to work

- Rooms of 4–5: your project team
- Blueprint template on campus (or a whiteboard)
- Use today's vocabulary: the nine steps, the three checks, the three environments, the three control families
- Nominate a spokesperson: 2 minutes



Output per room: one slide or whiteboard. Three rooms present in the debrief; everyone posts theirs on the forum.

The case: IberBank card-fraud model (facts only)

- The model scores every card transaction in real time (about 40 million a month) and must answer in under 150 ms. Above a threshold, the transaction is blocked. Three channels use it: point of sale, e-commerce, ATM.
- Features: the transaction (amount, merchant type, country), the customer profile from the warehouse (loaded nightly), and "velocity" features from a streaming system (spend in the last hour).
- Retrained quarterly by the data-science team on confirmed fraud cases. The fraud team labels about 2,000 cases a month; chargebacks arrive up to 60 days late.
- Incident 1 (March): a data vendor changed the unit of "amount" from euros to cents for a subset of merchants. For three days, thousands of legitimate purchases were blocked.
- Incident 2 (June): a fraud ring found that purchases just under €30 at grocery merchants were never blocked, and cycled 1,200 stolen cards through them over two weeks.
- Incident 3 (September): a "hotfix" to the threshold was pushed on a Friday evening from a data scientist's laptop straight to production, skipping QA. It also changed a feature calculation; false positives doubled over the weekend.

Blueprint template

- Threat (3 rows)
- Lifecycle stage where it enters
- One control
- Who owns the control
- How you would detect it: which lineage or monitoring signal
- Bonus: the regulator's view

Debrief: what we heard, and what it tells us

Incident	Enters at	Control	Owner	Detected by
1 • Vendor changed units	Ingestion / validation	Schema and range checks on the feed; data contract with the vendor	Data engineer	Validation report per span; blocked-rate spike per merchant
2 • Ring probes the threshold	Inference / feedback	Monitor probing patterns; no fixed public threshold; retrain on discovered patterns	ML engineer + fraud team	Amount-band and merchant patterns; late chargebacks confirm
3 • Friday hotfix from a laptop	Deployment	The two gates: versioned artefact, QA sign-off, rollback; no promotion from a laptop	Product owner (A), ML engineer (R), IT enforces	Lineage: a change with no QA record; weekend false-positive rate



Every control you named is a step in the pipeline or a gate between environments. That is what MLOps is.

Three things to remember

1

A notebook is a workshop; a pipeline is a factory.

Steps with inputs, outputs and checks, run by a machine in the same order every time. Validation stops the line before a bad model is built.

2

Version everything: code, data, models.

A repository, numbered data snapshots with a fingerprint, an experiment log and a registry. Any prediction can be traced back to its data.

3

Three environments, two gates.

Nothing goes from a laptop to production. Dev builds, QA tests as if live, Prod serves; someone accountable signs the gate.

Your forum this week: Session 4 • 10–16 October

Q1

Take the exploratory notebook provided on the virtual campus and sketch how you would restructure it into a reusable pipeline with separate ingestion, validation and preprocessing steps. Submit the diagram and a short rationale.

Q2

Explain why an organization needs separate Dev, QA and Prod environments. What could go wrong if a model is promoted straight from a data scientist's laptop to production?

Q3

Identify one adversarial or data-integrity threat to a deployed model and propose a mitigation. Which lineage or auditability control would help you detect it?



Opens today, closes Friday 16 October. No code needed for Q1: a drawing and a paragraph. Post early; reply to two classmates.

Before Session 5 (Saturday 17 October): read, and look at one thing



Read

- Treveil et al., Introducing MLOps, chapters 5–6: Preparing for production, Deploying to production
- Google Cloud, "MLOps: continuous delivery and automation pipelines in ML", up to MLOps level 1
- Optional: Huyen, Designing ML Systems, chapter 7



Look at one thing

- github.com/juanmaramirezgarcia/mlops-demo-churn → Actions tab
- Every push shows a green tick or a red cross. Write down what you think the tick is checking
- Session 5 starts from there

Next: Session 5 • CI/CD & Deployment Strategies

Tests for code and tests for data • how a model reaches customers without downtime: canary, blue-green, A/B • serverless hosting

Appendix • How models are tuned (not examinable)

Five slides for the curious: hyperparameters, search strategies, AutoML, how to challenge a model before trusting it, and what the MLflow lines look like in a training script. Not covered live; not in the exam.

Appendix • Hyperparameters: the settings that govern training



What they are

- Configuration variables set before training: number of trees, tree depth, learning rate, regularisation
- Not learned from the data; constant during a training job
- Each combination is a "trial": a complete training run



Why it matters for operations

- Every trial should be logged (the experiment log from the demo)
- The winning settings are part of the model artefact
- Retraining later must reuse or re-tune them deliberately

Appendix • Three ways to search: grid, random, Bayesian



Grid search

- A list of values per setting; try every combination
- Exhaustive, easy to understand, expensive



Random search

- Ranges per setting; sample combinations at random
- Usually finds good settings faster than grid



Bayesian optimisation

- Uses previous trials to pick the next candidate
- A surrogate model estimates performance cheaply
- Cold start: begins with random trials

Appendix • AutoML: automating the search, not the judgement



What AutoML does

- Selects algorithms and pre-processing automatically
- Searches hyperparameters; builds ensembles
- Estimates performance; sometimes explains the model
- Examples: cloud AutoML services, DataRobot, IBM AutoAI, auto-sklearn



What it does not do

- Decide what to predict, or whether the data may be used
- Define "good enough" for the business
- Operate the model afterwards: everything in this course
- Domain knowledge becomes more important, not less

Appendix • Before trusting a model, challenge it

1

Cross-check several metrics

Not only the one the model was optimised on. Accuracy can look fine while recall on the minority class collapses.

2

Split by subpopulation

Same error rate for men and women? for regions? for new and old customers? Session 7: fairness.

3

Compare with production

Is the new model better than the one already serving customers, on the same data? If not, do not deploy.

Appendix • The eight lines that give a training script a memory

A plain training script

```
df = read_csv("churn.csv")
validate(df)          # stop if wrong
X_train, X_test, y_train, y_test = split(df)

model = RandomForest(n_estimators=200,
                    max_depth=6)
model.fit(X_train, y_train)
accuracy = score(model, X_test, y_test)

save(model, "model.pkl")
print(accuracy)        # ...and then?
```

The same script, with a memory

```
import mlflow
mlflow.set_experiment("churn-model")
df = read_csv("churn.csv"); validate(df)
X_train, X_test, y_train, y_test = split(df)
with mlflow.start_run(): # one attempt
    model = RandomForest(n_estimators=200, max_depth=6)
    model.fit(X_train, y_train)
    accuracy = score(model, X_test, y_test)
    mlflow.log_params({"n_estimators": 200,
                      "max_depth": 6, "data_version": "06-01"})
    mlflow.log_metrics({"accuracy": accuracy})
    mlflow.set_tags({"owner": "Data Platform",
                    "requested_by": "Retention"})
    mlflow.sklearn.log_model(model, name="model")
    mlflow.register_model(..., "churn-model") # optional
```

Settings (what was tried) • metrics (how good) • tags (who, for whom, why) • the model file. The real file: S03/demo/mlflow/train_with_mlflow.py